🎨 UI Mockups & 🧠 Backend Architecture Guide

Rate My Teacher — Congressional App Challenge

This guide provides the complete blueprint for your team's frontend and backend developers:

1. **Visual UI Mockups (Wireframes)**: Visual layouts for `index.html`, `teacher.html`, and `add_teacher.html` (including empty states and review forms) designed for inspiration without writing HTML.
2. **Middle-Tier / Database Logic (`server/db_gateway.py`)**: Python functions that talk directly to SQLite.
3. **Web Server & REST APIs (`server/dispatcher.py`)**: Flask endpoints serving HTML templates and JSON endpoints with full error handling.

🎨 Part 1: Front-End UI Mockups (For Front-End Builders)

> 💡 **Tip for Front-End Developers:** Use these box-drawing wireframes as your visual guide when writing your HTML, CSS (`style.css`), and JavaScript (`app.js`).

```
================================================================================
MOCKUP 1: Main Landing Page (`website/templates/index.html`)
================================================================================

+-------------------------------------------------------------------------------+
|  🏫 Rate My Teacher            [🔍 Search by teacher, subject, school...]   [+ Add Teacher]  |
+-------------------------------------------------------------------------------+
|  🌟 Anonymous, safe & honest teacher ratings by students, for students!            |
+-------------------------------------------------------------------------------+
|                                                                               |
|  [ Filter by Grade: (All ▼) ]   [ Filter by Subject: (All ▼) ]   [ Sort: (Highest Rated ▼) ] |
|                                                                               |
|  Found 12 Teachers                                                            |
|  +-------------------------------------------------+---------------------------+ |
|  | 👨‍🏫 Mr. David Miller                              | ⭐⭐⭐⭐⭐ 4.8 / 5.0        | |
|  | Oak Creek Middle School • Los Angeles, CA       | 📊 Difficulty: 3.2 / 5.0   | |
|  | Subject: Mathematics | Grades: 6-7              | 💬 14 Reviews              | |
|  | "Middle School Pre-Algebra and Algebra 1"       |                           | |
|  | [Tag: Fun Projects] [Tag: Gives Good Feedback]  | [ View Reviews & Rate → ]  | |
|  +-------------------------------------------------+---------------------------+ |
|                                                                               |
|  +-------------------------------------------------+---------------------------+ |
|  | 👩‍🏫 Ms. Sarah Connor                              | ⭐⭐⭐⭐☆ 4.2 / 5.0        | |
|  | Oak Creek Middle School • Los Angeles, CA       | 📊 Difficulty: 4.1 / 5.0   | |
|  | Subject: Science | Grades: 7-8                  | 💬 9 Reviews               | |
|  | "7th and 8th Grade Physical & Life Sciences"    |                           | |
|  | [Tag: Tough Grader] [Tag: Inspirational]        | [ View Reviews & Rate → ]  | |
|  +-------------------------------------------------+---------------------------+ |
|                                                                               |
|  +-------------------------------------------------------------------------+ |
|  |  Can't find your teacher?                                                | |
|  |  [ + Submit a New Teacher Profile ]                                      | |
|  +-------------------------------------------------------------------------+ |
+-------------------------------------------------------------------------------+
```

```
================================================================================
MOCKUP 2: Teacher Details & Review Feed (`website/templates/teacher.html`)
================================================================================

+-----------------------------------------------------------------------------+
|  [ ← Back to Teacher List ]                          🏫 Rate My Teacher     |
+-----------------------------------------------------------------------------+
|                                                                             |
|  +-------------------------------------------------------------------------+ |
|  | 👨‍🏫 David Miller                                Overall Score:          | |
|  | Oak Creek Middle School • Los Angeles, CA       ⭐⭐⭐⭐⭐ 4.8 / 5.0      | |
|  | Department: Mathematics | Grade Levels: 6-7      Based on 14 student reviews | |
|  | Bio: "Middle School Pre-Algebra and Algebra 1"  Difficulty: 3.2 / 5.0    | |
|  |                                                 Helpfulness: 4.9 / 5.0   | |
|  | Top Tags: [Fun Projects (8)] [Gives Good Feedback (6)] [Clear Grading (5)] | |
|  +-------------------------------------------------------------------------+ |
|                                                                             |
|  REVIEWS (14)                                        [ ✍️ Write a Review ]   |
|  ---------------------------------------------------------------------------  |
|                                                                             |
|  +-------------------------------------------------------------------------+ |
|  | ⭐⭐⭐⭐⭐ Overall: 5 | Difficulty: 3 | Helpfulness: 5      Course: Algebra 1 | |
|  | Grade 7 Student • Fall Semester                                         | |
|  |                                                                         | |
|  | "Mr. Miller explains equations using pizza slices! His tests are fair if you do | |
|  |  the practice problems. Best math teacher ever."                        | |
|  |                                                                         | |
|  | Tags: [Fun Projects] [Gives Good Feedback]                               | |
|  | 👍 Helpful (12)    👎 (0)                                               | |
|  +-------------------------------------------------------------------------+ |
|                                                                             |
|  ---------------------------------------------------------------------------  |
|  ✍️ LEAVE AN ANONYMOUS REVIEW FOR MR. MILLER                                 |
|  ---------------------------------------------------------------------------  |
|  Course Name:        [ e.g. Pre-Algebra                    ]                |
|  Your Role:          (o) Student   ( ) Parent   ( ) Alumni                  |
|  Your Grade Level:   [ Grade 7 ▼ ]                                          |
|  Overall Rating:     [ ⭐ 1 ] [ ⭐ 2 ] [ ⭐ 3 ] [ ⭐ 4 ] [ ⭐ 5 ] (Click stars)          |
|  Difficulty:         [ 1 - Easy ] ... [ 3 - Medium ] ... [ 5 - Very Hard ]  |
|  Helpfulness:        [ 1 - Low  ] ... [ 3 - Moderate ] ... [ 5 - Super Helpful ] |
|  Select Tags:        [ ] Fun Projects    [ ] Lots of Homework    [ ] Tough Grader |
|                      [ ] Gives Good Feedback   [ ] Clear Criteria            |
|  Review Comments:                                                           |
|  +-------------------------------------------------------------------------+ |
|  | Write honest, constructive feedback here. Please follow community guidelines. | |
|  +-------------------------------------------------------------------------+ |
|  🔒 100% Anonymous. No email or name is recorded.                           |
|  [ 🚀 Submit Review ]                                                       |
+-----------------------------------------------------------------------------+
```

```
================================================================================
MOCKUP 2B: Empty State when a Teacher Has No Reviews Yet
================================================================================

+-----------------------------------------------------------------------------+
|  REVIEWS (0)                                                                |
|  ---------------------------------------------------------------------------  |
|  +-------------------------------------------------------------------------+ |
|  |                                                                         | |
|  |                          🌟 No Reviews Yet! 🌟                           | |
|  |                                                                         | |
|  |          Be the very first student or parent to share helpful feedback  | |
|  |                   for Ms. Sarah Connor in Science!                      | |
|  |                                                                         | |
|  |                          [ ✍️ Write First Review ]                       | |
|  |                                                                         | |
|  +-------------------------------------------------------------------------+ |
+-----------------------------------------------------------------------------+

================================================================================
MOCKUP 3: Add New Teacher Profile (`website/templates/add_teacher.html`)
================================================================================

+-----------------------------------------------------------------------------+
|  [ ⬅ Back to Teacher List ]                          🏫 Rate My Teacher     |
+-----------------------------------------------------------------------------+
|                                                                             |
|  ➕ Submit a New Teacher Profile                                            |
|  Help your school community by adding teachers so classmates can share tips! |
|                                                                             |
|  +-------------------------------------------------------------------------+ |
|  | School:                                                                 | |
|  | [ Oak Creek Middle School - Los Angeles, CA                          ▼ ] | |
|  |                                                                         | |
|  | Teacher First Name:                  Teacher Last Name:                 | |
|  | [ e.g. Sarah                       ]  [ e.g. Connor                   ] | |
|  |                                                                         | |
|  | Department / Subject:                Grade Level(s) Taught:             | |
|  | [ e.g. Science / Biology           ] [ e.g. 7th, 8th or K-5             ] | |
|  |                                                                         | |
|  | Short Bio or Description (Optional):                                    | |
|  | +---------------------------------------------------------------------+ | |
|  | | e.g. Teaches 8th grade Honors Physical Science and coaches Robotics Club. | | |
|  | +---------------------------------------------------------------------+ | |
|  |                                                                         | |
|  | [ ✅ Create Teacher Profile ]       [ Cancel ]                          | |
|  +-------------------------------------------------------------------------+ |
|                                                                             |
|  +-------------------------------------------------------------------------+ |
|  | ℹ️ Community Rule: Please make sure the teacher's name is spelled correctly. | |
|  +-------------------------------------------------------------------------+ |
+-----------------------------------------------------------------------------+
```

🧠 Part 2: Middle-Tier / Database Layer (`server/db_gateway.py`)

This file contains all SQL queries and helper functions. It connects to `database/app_database.db`, enforces foreign key constraints, calculates rating aggregates, and validates input data.

Save this file as **`server/db_gateway.py`**:

```
"""
Database Gateway for Rate My Teacher
Handles all SQLite queries, transactions, and aggregate calculations.
"""

import os
import sqlite3

# Locate app_database.db in the database/ directory relative to this file
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DB_PATH = os.path.join(BASE_DIR, "database", "app_database.db")


def get_db():
    """Establishes database connection with Row factory and Foreign Keys enabled."""
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON;")
    return conn


# ------------------------------------------------------------------------
# SCHOOL QUERIES
# ------------------------------------------------------------------------

def get_all_schools():
    """Returns all schools for dropdown selectors."""
    conn = get_db()
    cur = conn.cursor()
    cur.execute("SELECT * FROM schools ORDER BY name ASC")
    rows = [dict(row) for row in cur.fetchall()]
    conn.close()
    return rows


# ------------------------------------------------------------------------
# TEACHER QUERIES
# ------------------------------------------------------------------------

def get_all_teachers(search_query=None, department=None, grade_level=None):
    """
    Retrieves teachers with joined school info and computed review summary stats:
    - total_reviews
    - avg_overall (rounded to 1 decimal)
    - avg_difficulty (rounded to 1 decimal)
    """
    conn = get_db()
    cur = conn.cursor()

    sql = """
        SELECT
            t.teacher_id,
            t.school_id,
            t.first_name,
            t.last_name,
            t.department,
            t.grade_level,
            t.bio,
            t.created_at,
            s.name AS school_name,
            s.city AS school_city,
```

```
            s.state AS school_state,
            COUNT(r.review_id) AS total_reviews,
            ROUND(AVG(r.rating_overall), 1) AS avg_overall,
            ROUND(AVG(r.rating_difficulty), 1) AS avg_difficulty,
            ROUND(AVG(r.rating_helpfulness), 1) AS avg_helpfulness
        FROM teachers t
        JOIN schools s ON t.school_id = s.school_id
        LEFT JOIN reviews r ON t.teacher_id = r.teacher_id AND r.status = 'approved'
        WHERE 1=1
    """
    params = []

    if search_query:
        query_wildcard = f"%{search_query.strip()}%"
        sql += """ AND (
            t.first_name LIKE ? OR
            t.last_name LIKE ? OR
            t.department LIKE ? OR
            s.name LIKE ?
        )"""
        params.extend([query_wildcard, query_wildcard, query_wildcard, query_wildcard])

    if department:
        sql += " AND t.department = ?"
        params.append(department.strip())

    if grade_level:
        sql += " AND t.grade_level LIKE ?"
        params.append(f"%{grade_level.strip()}%")

    sql += """
        GROUP BY t.teacher_id
        ORDER BY t.last_name ASC, t.first_name ASC
    """

    cur.execute(sql, params)
    teachers = [dict(row) for row in cur.fetchall()]
    conn.close()
    return teachers


def get_teacher_by_id(teacher_id):
    """Returns detailed profile and aggregate scores for a specific teacher."""
    conn = get_db()
    cur = conn.cursor()
    cur.execute("""
        SELECT
            t.teacher_id,
            t.school_id,
            t.first_name,
            t.last_name,
            t.department,
            t.grade_level,
            t.bio,
            t.created_at,
            s.name AS school_name,
            s.city AS school_city,
            s.state AS school_state,
            COUNT(r.review_id) AS total_reviews,
            COALESCE(ROUND(AVG(r.rating_overall), 1), 0.0) AS avg_overall,
            COALESCE(ROUND(AVG(r.rating_difficulty), 1), 0.0) AS avg_difficulty,
```

```
            COALESCE(ROUND(AVG(r.rating_helpfulness), 1), 0.0) AS avg_helpfulness
        FROM teachers t
        JOIN schools s ON t.school_id = s.school_id
        LEFT JOIN reviews r ON t.teacher_id = r.teacher_id AND r.status = 'approved'
        WHERE t.teacher_id = ?
        GROUP BY t.teacher_id
    """, (teacher_id,))
    row = cur.fetchone()
    teacher = dict(row) if row else None
    conn.close()
    return teacher


def add_teacher(school_id, first_name, last_name, department, grade_level, bio=""):
    """
    Inserts a new teacher into the teachers table.
    Returns (success: bool, teacher_id: int or None, error: str or None).
    """
    if not school_id or not first_name or not last_name or not department or not grade_level:
        return False, None, "All fields (School, First Name, Last Name, Department, Grade Level) are required.

    conn = get_db()
    cur = conn.cursor()
    try:
        cur.execute("""
            INSERT INTO teachers (school_id, first_name, last_name, department, grade_level, bio)
            VALUES (?, ?, ?, ?, ?, ?)
        """, (
            int(school_id),
            first_name.strip(),
            last_name.strip(),
            department.strip(),
            grade_level.strip(),
            (bio or "").strip()
        ))
        teacher_id = cur.lastrowid
        conn.commit()
        return True, teacher_id, None
    except sqlite3.IntegrityError as e:
        return False, None, f"Database integrity error: {str(e)}"
    except Exception as e:
        return False, None, f"Failed to add teacher: {str(e)}"
    finally:
        conn.close()


# ---------------------------------------------------------------------------
# REVIEW QUERIES
# ---------------------------------------------------------------------------

def get_teacher_reviews(teacher_id, current_client_hash=None):
    """
    Retrieves all approved reviews for a teacher, including attached tags,
    helpful vote counts, and user's previous vote status.
    """
    conn = get_db()
    cur = conn.cursor()

    cur.execute("""
        SELECT
            r.review_id,
```

```
            r.teacher_id,
            r.course_name,
            r.reviewer_role,
            r.student_grade_level,
            r.rating_overall,
            r.rating_difficulty,
            r.rating_helpfulness,
            r.review_text,
            r.created_at,
            COALESCE(SUM(CASE WHEN v.vote_value = 1 THEN 1 ELSE 0 END), 0) AS upvotes,
            COALESCE(SUM(CASE WHEN v.vote_value = -1 THEN 1 ELSE 0 END), 0) AS downvotes,
            MAX(CASE WHEN v.client_fingerprint_hash = ? THEN v.vote_value ELSE 0 END) AS user_vote
        FROM reviews r
        LEFT JOIN review_votes v ON r.review_id = v.review_id
        WHERE r.teacher_id = ? AND r.status = 'approved'
        GROUP BY r.review_id
        ORDER BY r.created_at DESC
    """, (current_client_hash or "", teacher_id))

    reviews = [dict(row) for row in cur.fetchall()]

    # Fetch tags for each review
    for rev in reviews:
        cur.execute("""
            SELECT t.tag_id, t.tag_name
            FROM tags t
            JOIN review_tags rt ON t.tag_id = rt.tag_id
            WHERE rt.review_id = ?
            ORDER BY t.tag_name ASC
        """, (rev["review_id"],))
        rev["tags"] = [dict(t) for t in cur.fetchall()]

    conn.close()
    return reviews


def add_review(teacher_id, rating_overall, rating_difficulty, rating_helpfulness,
               review_text, client_fingerprint_hash, course_name=None,
               reviewer_role="student", student_grade_level=None, tag_ids=None):
    """
    Saves a new review.
    Enforces 100% anonymous rate-limiting: one review per (teacher, course, client_fingerprint_hash).
    Returns (success: bool, review_id: int or None, error: str or None).
    """
    # Validation
    if not teacher_id:
        return False, None, "Teacher ID is missing."
    if not (1 <= int(rating_overall) <= 5):
        return False, None, "Overall rating must be between 1 and 5."
    if not (1 <= int(rating_difficulty) <= 5):
        return False, None, "Difficulty rating must be between 1 and 5."
    if not (1 <= int(rating_helpfulness) <= 5):
        return False, None, "Helpfulness rating must be between 1 and 5."
    if not review_text or len(review_text.strip()) < 5:
        return False, None, "Please write at least a few words for your review comments."

    conn = get_db()
    cur = conn.cursor()
    try:
        cur.execute("""
            INSERT INTO reviews (
```

```
                teacher_id, course_name, reviewer_role, student_grade_level,
                rating_overall, rating_difficulty, rating_helpfulness,
                review_text, status, client_fingerprint_hash
            )
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'approved', ?)
        """, (
            int(teacher_id),
            (course_name or "").strip() or None,
            reviewer_role,
            int(student_grade_level) if student_grade_level else None,
            int(rating_overall),
            int(rating_difficulty),
            int(rating_helpfulness),
            review_text.strip(),
            client_fingerprint_hash
        ))
        review_id = cur.lastrowid

        # Insert selected tags
        if tag_ids and isinstance(tag_ids, list):
            for t_id in tag_ids:
                cur.execute("""
                    INSERT OR IGNORE INTO review_tags (review_id, tag_id)
                    VALUES (?, ?)
                """, (review_id, int(t_id)))

        conn.commit()
        return True, review_id, None

    except sqlite3.IntegrityError:
        return False, None, "You have already submitted a review for this teacher and course."
    except Exception as e:
        return False, None, f"Failed to save review: {str(e)}"
    finally:
        conn.close()


def get_all_tags():
    """Returns all pre-seeded review tags (e.g. Fun Projects, Tough Grader)."""
    conn = get_db()
    cur = conn.cursor()
    cur.execute("SELECT * FROM tags ORDER BY tag_name ASC")
    tags = [dict(row) for row in cur.fetchall()]
    conn.close()
    return tags
```

🚀 Part 3: Middle-Tier Web Server & APIs (`server/dispatcher.py`)

This file runs the Flask web server. It handles:

- **HTML Pages**: Serves `index.html`, `teacher.html`, and `add_teacher.html`.
- **REST APIs**: Returns JSON data for dynamic JavaScript interactions with clear status codes (`200 OK`, `201 Created`, `400 Bad Request`, `404 Not Found`).
- **Anonymous Client Tracking**: Generates secure session hashes to prevent spam without collecting any personal data.

Save this file as **`server/dispatcher.py`**:

```
"""
Flask Dispatcher Server for Rate My Teacher
Routes requests between the browser (Front-End) and the database gateway (Back-End).
"""

import hashlib
import os
import secrets
from flask import Flask, jsonify, render_template, request, session
import db_gateway

# Configure paths to the website folder
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
TEMPLATE_DIR = os.path.join(BASE_DIR, "website", "templates")
STATIC_DIR = os.path.join(BASE_DIR, "website", "static")

dispatcher = Flask(
    __name__,
    template_folder=TEMPLATE_DIR,
    static_folder=STATIC_DIR
)
dispatcher.secret_key = "rate-my-teacher-anonymous-development-key"


def get_client_hash():
    """
    Creates or retrieves an anonymous, cryptographic session token.
    Complies with COPPA: no IP addresses, usernames, or cookies with personal data are stored.
    """
    if "anonymous_user_token" not in session:
        session["anonymous_user_token"] = secrets.token_hex(16)
    return hashlib.sha256(session["anonymous_user_token"].encode()).hexdigest()


# ============================================================================
# 🌐 HTML PAGE ROUTES
# ============================================================================

@dispatcher.route("/")
def index_page():
    """Main landing page: lists all teachers with optional search and filter."""
    query = request.args.get("q", "")
    department = request.args.get("department", "")
    grade = request.args.get("grade", "")

    teachers = db_gateway.get_all_teachers(query, department, grade)
    schools = db_gateway.get_all_schools()

    return render_template(
        "index.html",
        teachers=teachers,
        schools=schools,
        search_query=query,
        selected_department=department,
        selected_grade=grade
    )


@dispatcher.route("/teacher/<int:teacher_id>")
def teacher_page(teacher_id):
    """Teacher details page: displays profile, rating summary, and review feed."""
```

```
    teacher = db_gateway.get_teacher_by_id(teacher_id)
    if not teacher:
        return "<h1>Teacher not found</h1><p><a href='/'>Return to Home</a></p>", 404

    client_hash = get_client_hash()
    reviews = db_gateway.get_teacher_reviews(teacher_id, client_hash)
    available_tags = db_gateway.get_all_tags()

    return render_template(
        "teacher.html",
        teacher=teacher,
        reviews=reviews,
        tags=available_tags,
        has_reviews=len(reviews) > 0
    )


@dispatcher.route("/add-teacher")
def add_teacher_page():
    """Page with the form to add a new teacher profile."""
    schools = db_gateway.get_all_schools()
    return render_template("add_teacher.html", schools=schools)


# ============================================================================
# 🔌 REST API ENDPOINTS (For JavaScript Fetch / Async Requests)
# ============================================================================

@dispatcher.route("/api/teachers", methods=["GET"])
def api_get_teachers():
    """API to fetch teachers list as JSON."""
    query = request.args.get("q")
    department = request.args.get("department")
    grade = request.args.get("grade")

    teachers = db_gateway.get_all_teachers(query, department, grade)
    return jsonify({"success": True, "count": len(teachers), "teachers": teachers}), 200


@dispatcher.route("/api/teachers", methods=["POST"])
def api_create_teacher():
    """
    API to create a new teacher.
    Accepts JSON or form data.
    """
    data = request.get_json() if request.is_json else request.form

    school_id = data.get("school_id")
    first_name = data.get("first_name")
    last_name = data.get("last_name")
    department = data.get("department")
    grade_level = data.get("grade_level")
    bio = data.get("bio", "")

    success, teacher_id, error = db_gateway.add_teacher(
        school_id, first_name, last_name, department, grade_level, bio
    )

    if not success:
        return jsonify({"success": False, "error": error}), 400
```

```
    return jsonify({
        "success": True,
        "message": "Teacher profile created successfully!",
        "teacher_id": teacher_id
    }), 201


@dispatcher.route("/api/teachers/<int:teacher_id>", methods=["GET"])
def api_get_teacher_detail(teacher_id):
    """API to fetch single teacher details."""
    teacher = db_gateway.get_teacher_by_id(teacher_id)
    if not teacher:
        return jsonify({"success": False, "error": "Teacher not found"}), 404
    return jsonify({"success": True, "teacher": teacher}), 200


@dispatcher.route("/api/teachers/<int:teacher_id>/reviews", methods=["GET"])
def api_get_reviews(teacher_id):
    """API to fetch all reviews for a teacher."""
    client_hash = get_client_hash()
    reviews = db_gateway.get_teacher_reviews(teacher_id, client_hash)
    return jsonify({
        "success": True,
        "teacher_id": teacher_id,
        "count": len(reviews),
        "reviews": reviews
    }), 200


@dispatcher.route("/api/teachers/<int:teacher_id>/reviews", methods=["POST"])
def api_submit_review(teacher_id):
    """
    API to submit an anonymous review for a teacher.
    Accepts JSON or form data.
    """
    data = request.get_json() if request.is_json else request.form

    rating_overall = data.get("rating_overall")
    rating_difficulty = data.get("rating_difficulty")
    rating_helpfulness = data.get("rating_helpfulness")
    review_text = data.get("review_text")
    course_name = data.get("course_name")
    reviewer_role = data.get("reviewer_role", "student")
    student_grade_level = data.get("student_grade_level")
    tag_ids = data.get("tag_ids", [])

    client_hash = get_client_hash()

    success, review_id, error = db_gateway.add_review(
        teacher_id=teacher_id,
        rating_overall=rating_overall,
        rating_difficulty=rating_difficulty,
        rating_helpfulness=rating_helpfulness,
        review_text=review_text,
        client_fingerprint_hash=client_hash,
        course_name=course_name,
        reviewer_role=reviewer_role,
        student_grade_level=student_grade_level,
        tag_ids=tag_ids
    )
```

```
    if not success:
        return jsonify({"success": False, "error": error}), 400

    return jsonify({
        "success": True,
        "message": "Thank you! Your anonymous review has been posted.",
        "review_id": review_id
    }), 201


@dispatcher.route("/api/schools", methods=["GET"])
def api_get_schools():
    """API to fetch all schools."""
    schools = db_gateway.get_all_schools()
    return jsonify({"success": True, "schools": schools}), 200


@dispatcher.route("/api/tags", methods=["GET"])
def api_get_tags():
    """API to fetch all pre-defined tags."""
    tags = db_gateway.get_all_tags()
    return jsonify({"success": True, "tags": tags}), 200


# ============================================================================
# 🏁 APPLICATION ENTRY POINT
# ============================================================================

if __name__ == "__main__":
    print("🚀 Rate My Teacher Web Server running at http://127.0.0.1:5000")
    dispatcher.run(debug=True, port=5000)
```